

Lab Manual

for

Network Programming

Diploma In Computer Hardware Engineering

5th Semester

By

SITTTR

Kalamassery

GENERAL INSTRUCTIONS

Rough record and Fair record are needed to record the experiments conducted in the laboratory. Rough records are needed to be certified immediately on completion of the experiment. Fair records are due at the beginning of the next lab period. Fair records must be submitted as neat, legible, and complete.

INSTRUCTIONS TO STUDENTS FOR WRITING THE FAIR RECORD

In the fair record, the index page should be filled properly by writing the corresponding experiment number, experiment name, date on which it was done and the page number.

On the **right side** page of the record following has to be written:

1. **Title:** The title of the experiment should be written in the page in capital letters.
2. In the left top margin, experiment number and date should be written.
3. **Aim:** The purpose of the experiment should be written clearly.
4. **Apparatus/Tools/Equipments/Components used:** A list of the Apparatus/Tools /Equipments /Components used for doing the experiment should be entered.
5. **Principle:** Simple working of the circuit/experimental set up/algorithm should be written.
6. **Procedure:** steps for doing the experiment and recording the readings should be briefly described(flow chart/programs in the case of computer/processor related experiments)
7. **Results:** The results of the experiment must be summarized in writing and should be fulfilling the aim.
8. **Inference :** Inference from the results is to be mentioned.

On the **Left side** page of the record following has to be recorded:

1. **Circuit/Program:** Neatly drawn circuit diagrams/experimental set up.
2. **Design:** The design of the circuit/experimental set up for selecting the components should be clearly shown if necessary.

3. **Observations:** i) Data should be clearly recorded using Tabular Columns.

ii) Unit of the observed data should be clearly mentioned

iii) Relevant calculations should be shown. If repetitive calculations are needed, only show a sample calculation and summarize the others in a table.

4. **Graphs :** Graphs can be used to present data in a form that shows the results obtained, as one or more of the parameters are varied. A graph has the advantage of presenting large

amounts of data in a concise visual form. Graph should be in a square format.

GENERAL RULES FOR PERSONAL SAFETY

1. Always wear tight shirt/lab coat , pants and shoes inside workshops.

2. REMOVE ALL METAL JEWELLERY since rings, wrist watches or bands, necklaces, etc. make excellent electrodes in the event of accidental contact with electric power sources.

3. DO NOT MAKE CIRCUIT CHANGES without turning off the power.

4. Make sure that equipment working on electrical power are grounded properly.

5. Avoid standing on metal surfaces or wet concrete. Keep your shoes dry.

6. Never handle electrical equipment with wet skin.

7. Hot soldering irons should be rested in its holder. Never leave a hot iron unattended.

8. Avoid use of loose clothing and hair near machines and avoid running around inside lab .

TO PROTECT EQUIPMENT AND MINIMIZE MAINTENANCE:

DO: 1. SET MULTIRANGE METERS to highest range before connecting to an unknown source.

2. INFORM YOUR INSTRUCTOR about faulty equipment so that it can be sent for repair.

DO NOT: 1. Do not MOVE EQUIPMENT around the room except under the supervision of an instructor.

INDEX

<u>NO</u>	<u>LIST OF EXPERIMENTS</u>	<u>PAGE NO</u>
1	IMPLEMENTATION OF INHERITANCE	3
2	IMPLEMENTATION OF INTERFACE	7
3	IMPLEMENTATION OF THREAD	9
4	IMPLEMENTATION OF GUI WITH AWT	13
5	PROGRAM USING TCP SOCKETS DATE AND TIME SERVER	18
6	IMPLEMENTATION OF CLIENT SERVER COMMUNICATION USING TCP	23
7	IMPLEMENTATION OF TCP/IP ECHO	27
8	PROGRAM USING UDP SOCKET UDP CHAT SERVER/CLIENT	33
9	UDP DNS SERVER/CLIENT	39
10	ARITHMETIC CALCULATOR USING RPC-RMI	45

Experiment No:1
IMPLEMENTATION OF INHERITANCE

AIM: Write a Java Program to implement multilevel inheritance by applying various access controls to its data members and methods.

DESCRIPTION:

The base class Student provides methods to read details of students. The class Marks is inherited from Student which possesses methods to accept and prints marks of student. Class Result is derived from class Marks which computes the result.

ALGORITHM:

1. Create base class Student and define its methods.
2. Derive class Marks from Student and define its data members and methods
3. Derive subclass Result from Marks and define its data members and methods.
4. Create objects of class Result.
5. Invoke appropriate methods using the created object.
6. Stop.

PROGRAM

```
import java.io.DataInputStream;
class Student
{
    private int rollNo;
    private String name;
    DataInputStream dis=new DataInputStream(System.in);
    public void getrollNo()
    {
        try
```

```

{
System.out.println("Enter rollno ");
rollno=Integer.parseInt(dis.readLine());
System.out.println("Enter name ");
name=dis.readLine();
}
catch(Exception e){ }
}
void putrollno()
{
System.out.println("Roll No =" +rollno);
System.out.println("Name =" +name);
}
}
class Marks extends Student
{
protected int m1,m2,m3;
void getmarks()
{
try
{
System.out.println("Enter marks :");
m1=Integer.parseInt(dis.readLine());
m2=Integer.parseInt(dis.readLine());
m3=Integer.parseInt(dis.readLine());
}
catch(Exception e) { }
}
void putmarks()
{
System.out.println("m1="+m1);
System.out.println("m2="+m2);
}
}

```

```

System.out.println("m3="+m3);
}
}
class Result extends Marks
{
private float total;
void compute_display()
{
total=m1+m2+m3;
System.out.println("Total marks :"+total);
}
}
class Multilevel
{
public static void main(String arg[])
{
Result r=new Result();
r.getrollno();
r.getmarks();
r.putrollno();
r.putmarks();
r.compute_display();
}
}

```

SAMPLE OUTPUT

C:\jdk1.6.0_26\bin>javac Multilevel.java

Note: Multilevel.java uses or overrides a deprecated API.

Note: Recompile with -Xlint:deprecation for details.

C:\jdk1.6.0_26\bin>java Multilevel

Enter rollno

12345

Enter name

Avinash

Enter marks :

54

78

46

Roll No =12345

Name =Avinash

m1=54

m2=78

m3=46

Total marks :178.0

Experiment No:2

IMPLEMENTATION OF INTERFACE

AIM: Write a program to demonstrate use of implementing interfaces.

DESCRIPTION:

The interface acts as the base which only declares the methods data members.

The class which implements interface defines the methods. Then create objects of the implemented class and invoke the desired methods.

ALGORITHM:

1. Create an interface Area and declare its methods.
2. Create classs rectangle and circle which implements interface Area and define its data members and methods
3. Create objects of the classes created .
4. Invoke appropriate methods using the created objects.
5. Stop.

PROGRAM:

```
import java.lang.*;

interface Area
{
    final static float pi=3.14F;
    float compute(float x,float y);
}

class rectangle implements Area
{
    public float compute(float x,float y)
    {
```

```

return(pi*x*y);
}
}

class circle implements Area
{
public float compute(float x,float x)
{
return(pi*x*x);
}
}

class interfacedemo
{
public static void main(String a[])
{
rectangle rect=new rectangle();
circle cir=new circle();
Area A;
A=rect;
System.out.println("Area of rectangle="+A.compute(10,20));
A=cir;
System.out.println("Area of circle="+A.compute(30,0));
}
}

```

SAMPLE OUTPUT

C:\jdk1.6.0_26\bin>javac interfacedemo.java

C:\jdk1.6.0_26\bin>java interfacedemo

Area of rectangle=628.0

Area of circle=2,827.43

Experiment No:3
IMPLEMENTATION OF THREAD

AIM: Write a Java program that creates three threads. First thread displays “Good Morning” every one second, the second thread displays “Hello” every two seconds and the third thread displays “Welcome” every three seconds.

DESCRIPTION:

The threads in java implements multitasking. Create a thread by extending Thread class or by using Runnable interface. All threads contain an overridden run method. Create the objects of each class and start thread execution by calling the start method.

ALGORITHM:

1. Create three classes by extending the Thread class.
2. Override the run () method of Thread class in each sub class created to implement the desired operations.
3. Create the objects of each class created.
4. Start execution of each thread by calling the start () method using the created objects.
5. Stop.

PROGRAM

```
import java.io.*;

class One extends Thread
{
    public void run()
    {
        for(int i=0;i<100;i++)
        {
            try{
```

```

Thread.sleep(1000); }
catch(InterruptedException e){
System.out.println(e); }
System.out.println("Good Morning");
}
}
}
class Two extends Thread
{
public void run()
{
for(int i=0;i<100;i++)
{
try{
Thread.sleep(2000); }
catch(InterruptedException e)
{
System.out.println(e);
}
System.out.println("Hello ");
}
}
}
class Three implements Runnable
{
public void run()
{
for(int i=0;i<100;i++)
{
try{
Thread.sleep(3000); }

```

```

catch(InterruptedException e){
System.out.println(e); }
System.out.println("Wel come");
}
}
}
class ThreadEx
{
public static void main(String[] args)
{
One t1=new One();
Two t2=new Two();

Three tt=new Three();
Thread t3=new Thread(tt);
t1.setName("One");
t2.setName("Two");
t3.setName("Three");
System.out.println(t1);
System.out.println(t2);
System.out.println(t3);
Thread t=Thread.currentThread();
System.out.println(t);
t1.start();t2.start();t3.start();
}
}

```

SAMPLE OUTPUT:

C:\>javac ThreadEx.java

C:\>java ThreadEx

Thread[One,5,main]

Thread[Two,5,main]

Thread[Three,5,main]

Thread[main,5,main]

Good Morning

Good Morning

Hello

Wel come

Good Morning

Hello

Good Morning

Good Morning

Hello

Wel come

Good Morning

Good Morning

Hello

Good Morning

Experiment No:4
IMPLEMENTATION OF GUI WITH AWT

AIM: Write a program that creates a user interface to perform integer divisions. The user enters two numbers in the text fields, Num1 and Num2. The division of Num1 and Num2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw Number Format Exception. If Num2 were Zero, the program would throw an Arithmetic Exception Display the exception in a message dialog box.

DESCRIPTION:

User interface can be created in java with AWT packages. It is equipped with different controls to create layout of the desired user interface. There are numerous methods to create and manage windows.

ALGORITHM:

1. Create a subclass class inheriting Frame class and implementing ActionListener.
2. Create desired labels, buttons, text fields, etc. and add them to the created frame.
3. Map the methods to appropriate actions of the respected control objects.
4. Write the statements for paint () method.
5. Write statements to handle exceptions.
6. Run the program and check for desired output.
7. Stop.

PROGRAM

```
import java.awt.*;
import java.awt.event.*;
class NumDiv extends Frame implements ActionListener
{
    TextField num1, num2, res;
    Button division;
```

```

String msg=" ";
Label l1,l2;
NumDiv()
{
l1=new Label("Enter First Number");
l2=new Label("Enter Second Number");
num1=new TextField(30);
num2=new TextField(30);
res=new TextField(100);
division=new Button("Division");
setLayout(null);
setForeground(Color.blue);
setFont(new Font("Ariel", Font.BOLD,15));
l1.setBounds(100,100,200,30);
l2.setBounds(100,150,200,30);
num1.setBounds(350,100,100,30);
num2.setBounds(350,150,100,30);
division.setBounds(200,200,100,30);
res.setBounds(100,250,300,30);
add(l1); add(num1);
add(l2); add(num2);
add(res);
add(division);
setSize(600,600);
setVisible(true);
setTitle("Division");
setBackground(new Color(250,160,250));
addWindowListener(new WindowAdapter()
{
public void windowClosing(WindowEvent e)
{

```



```

System.exit(0);
}
});
division.addActionListener(this);
}
public void paint(Graphics g)
{
g.setColor(Color.black);
g.setFont(new Font("Arial",Font.BOLD,30));
}
public static void main(String[] args)
{
new NumDiv();
}
public void actionPerformed(ActionEvent e)
{
msg="";
int n1=1;int n2=1;int temp=1;
String s1=num1.getText();
String s2=num2.getText();
try
{
n1=Integer.parseInt(s1);
n2=Integer.parseInt(s2);
}
catch(NumberFormatException ex)
{
MyDialog d=new MyDialog(this, "NumberFormatException Dialog",ex);
}
if(n2==0)
{

```

```

try{ temp=n1/n2; } catch(ArithmeticException e1) { MyDialog d=new
MyDialog(this,"ArithmeticExceptionDialog",e1); }
}
else
{
msg+=n1/n2;
}
res.setText(msg);
}
}
class MyDialog extends Dialog
{
MyDialog(Frame f, String s, Exception e)
{
super(f,s,false);
setVisible(true);
setSize(500,500);
setForeground(Color.blue);
setFont(new Font("Arial",Font.BOLD,15));
setLayout(null);
Label l=new Label(e.toString());
l.setBounds(20,100,500,30);
add(l);
addWindowListener(new WindowAdapter()
{
public void windowClosing(WindowEvent e)
{
dispose();
}
}
}
}

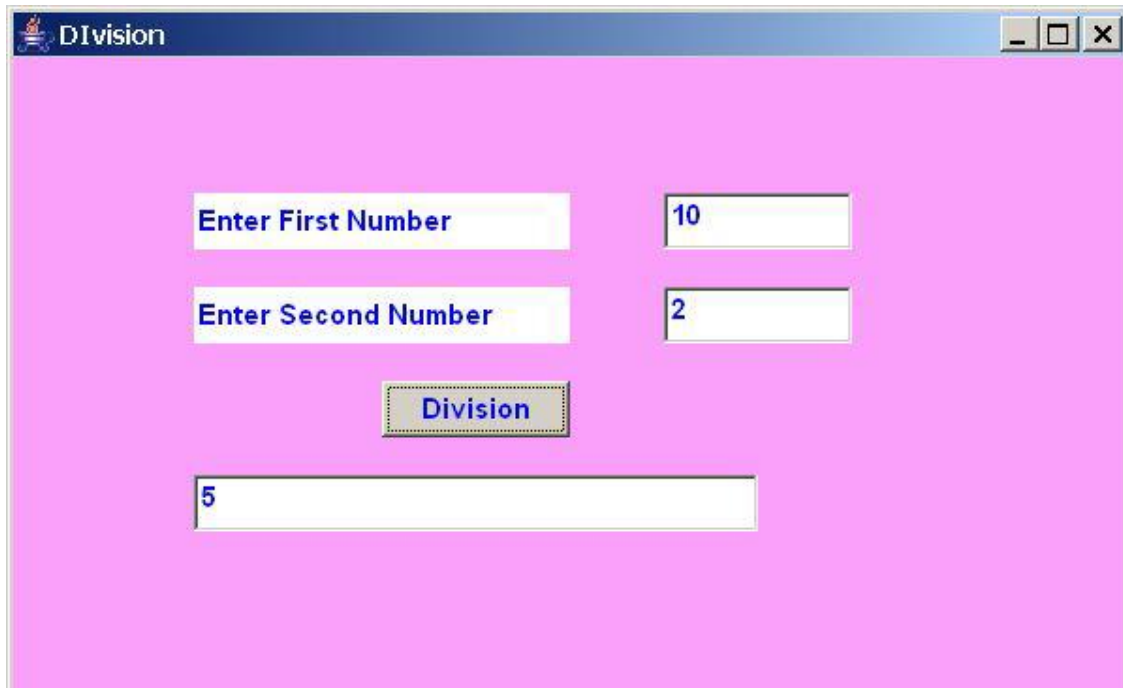
```

}

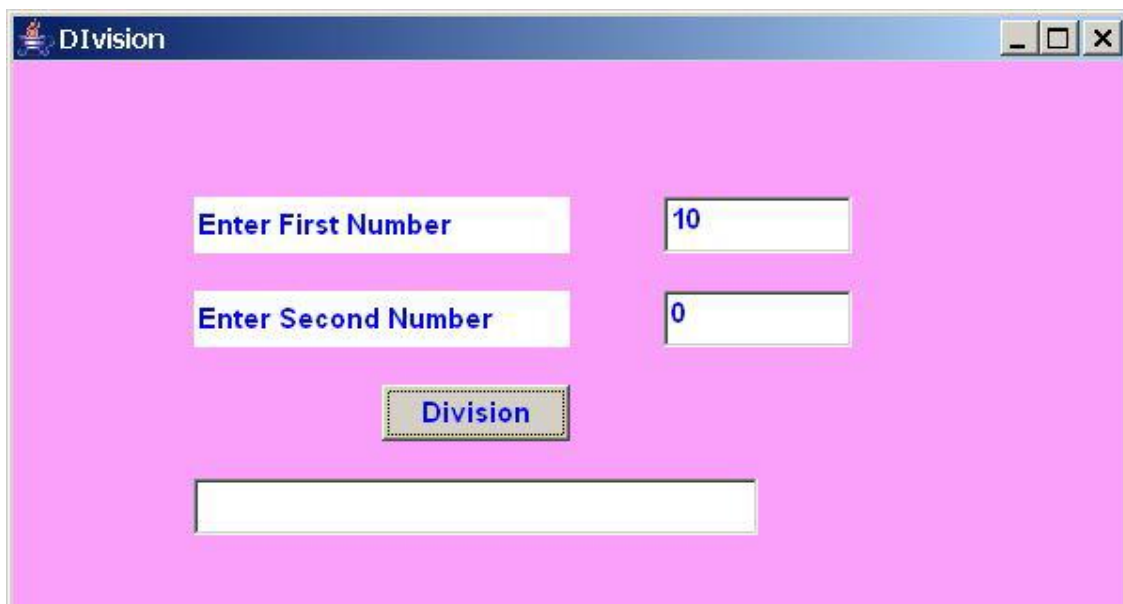
SAMPLE OUTPUT:

D:\Lab>javac NumDiv.java

D:\Lab>java NumDiv



The screenshot shows a Java application window titled "DDivision". The window has a pink background. It contains two input fields with labels "Enter First Number" and "Enter Second Number". The first input field contains the value "10" and the second contains "2". Below these is a button labeled "Division". At the bottom, there is a text area displaying the result "5".



The screenshot shows the same "DDivision" application window. The input fields now contain "10" and "0". The "Division" button is still present. The text area at the bottom is empty, indicating an error state where division by zero is attempted.



Experiment No:5

PROGRAM USING TCP SOCKETS DATE AND TIME SERVER

AIM: To implement date and time display from client to server using TCP Sockets

DESCRIPTION: TCP Server gets the system date and time and opens the server socket to read the client details. Client send its address to the server. Then client receives the date and time from server to display. TCP socket server client connection is opened for communication. After the date time is displayed the server client connection is closed with its respective streams to be closed.

ALGORITHM:

Server

1. Create a server socket and bind it to port.
2. Listen for new connection and when a connection arrives, accept it.
3. Send server's date and time to the client.
4. Read client's IP address sent by the client.
5. Display the client details.
6. Repeat steps 2-5 until the server is terminated.
7. Close all streams.
8. Close the server socket.
9. Stop.

Client

1. Create a client socket and connect it to the server's port number.
2. Retrieve its own IP address using built-in function.
3. Send its address to the server.
4. Display the date & time sent by the server.
5. Close the input and output streams.
6. Close the client socket.
7. Stop.

PROGRAM:

//TCP Date Server--tcpdateserver.java

```
import java.net.*;
```

```
import java.io.*;
```

```
import java.util.*;
```

```
class tcpdateserver
```

```
{
```

```
    public static void main(String arg[])
```

```
    {
```

```
        ServerSocket ss = null;
```

```
        Socket cs;
```

```
        PrintStream ps;
```

```
        BufferedReader dis;
```

```
        String inet;
```

```
        Try
```

```

{

ss = new ServerSocket(4444);

System.out.println("Press Ctrl+C to quit");

while(true)

{

cs = ss.accept();

ps = new PrintStream(cs.getOutputStream());

Date d = new Date();

ps.println(d);

dis = new BufferedReader(new InputStreamReader(cs.getInputStream()));

inet = dis.readLine();

System.out.println("Client System/IP address is :"+ inet);

ps.close();

dis.close();

}

}

catch(IOException e)

{

System.out.println("The exception is :"+ e);

}

}

}

```

```
// TCP Date Client--tcpdateclient.java

import java.net.*;

import java.io.*;

class tcpdateclient

{

public static void main (String args[])

{

Socket soc;

BufferedReader dis;

String sdate;

PrintStream ps;

Try

{

InetAddress ia = InetAddress.getLocalHost();

if (args.length == 0)

soc = new Socket(InetAddress.getLocalHost(),4444);

else

soc = new Socket(InetAddress.getByName(args[0]),4444);

dis = new BufferedReader(new InputStreamReader(soc.getInputStream()));

sdate=dis.readLine();

System.out.println("The date/time on server is : " +sdate);

ps = new PrintStream(soc.getOutputStream());
```

```
ps.println(ia);

ps.close();

catch(IOException e)

{

System.out.println("THE EXCEPTION is : " + e);

}

}

}
```

SAMPLE OUTPUT

Server:

```
$ javac tcpdateserver.java
```

```
$ java tcpdateserver
```

Press Ctrl+C to quit

Client System/IP address is : localhost.localdomain/127.0.0.1

Client System/IP address is : localhost.localdomain/127.0.0.1

Client:

```
$ javac tcpdateclient.java
```

```
$ java tcpdateclient
```

The date/time on server is: Wed Jul 06 07:12:03 GMT 2011

Every time when a client connects to the server, server's date/time will be returned to the client for synchronization.

Experiment No:6

Implementation of Client-Server Communication Using TCP

AIM:

To implement a chat server and client in java using TCP sockets.

DESCRIPTION:

TCP Clients sends request to server and server will receives the request and response with acknowledgement. Every time client communicates with server and receive response from it.

ALGORITHM:

Server

1. Create a server socket and bind it to port.
2. Listen for new connection and when a connection arrives, accept it.
3. Read Client's message and display it
4. Get a message from user and send it to client
5. Repeat steps 3-4 until the client sends "end"
6. Close all streams
7. Close the server and client socket
8. Stop

Client

1. Create a client socket and connect it to the server's port number

2. Get a message from user and send it to server
3. Read server's response and display it
4. Repeat steps 2-3 until chat is terminated with "end" message
5. Close all input/output streams
6. Close the client socket
7. Stop

PROGRAM:

```
//Server.java

import java.io.*;

import java.net.*;

class Server

{

public static void main(String args[])

{

String data = "Networks Lab";

Try

{

ServerSocket svr = new ServerSocket(1234);

Socket skt = svr.accept();

System.out.print("Server has connected!\n");

PrintWriter out = new PrintWriter(skt.getOutputStream(), true);

System.out.print("Sending string: " + data + "\n");
```

```

out.print(data);

out.close();

skt.close();

srvr.close();

}

catch(Exception e)

{

System.out.print("Whoops! It didn't work!\n");

}

}

}

//Client.java

import java.io.*;

import java.net.*;

class Client

{

public static void main(String args[])

{

Try

{

Socket skt = new Socket("localhost", 1234);

BufferedReader in = new BufferedReader(new InputStreamReader(skt.getInputStream()));

```

```
System.out.print("Received string: ");

while (!in.ready())

{

}

System.out.println(in.readLine());

System.out.print("\n"); in.close();

}

catch(Exception e)

{

System.out.print("Whoops! It didn't work!\n");

}

}

}
```

SAMPLE OUTPUT

Server:

\$ javac Server.java

\$ java Server

Server started

Client connected

Client

\$ javac Client.java

\$ java Client

RESULT

Thus both the client and server exchange data using TCP socket programming.

Experiment No:7

IMPLEMENTATION OF TCP/IP ECHO

AIM:

To implementation of echo client server using TCP/IP

DESCRIPTION:

TCP Server gets the message and opens the server socket to read the client details. Client send its address to the server. Then client receives the message from server to display.

ALGORITHM

Server

1. Create a server socket and bind it to port.
2. Listen for new connection and when a connection arrives, accept it.
3. Read the data from client.
4. Echo the data back to the client.
5. Repeat steps 4-5 until „bye“ or „null“ is read.
6. Close all streams.
7. Close the server socket.
8. Stop.

Client

1. Create a client socket and connect it to the server's port number.
2. Get input from user.

3. If equal to bye or null, then go to step 7.
4. Send user data to the server.
5. Display the data echoed by the server.
6. Repeat steps 2-4.
7. Close the input and output streams.
8. Close the client socket.
9. Stop.

PROGRAM:

```
// TCP Echo Server--tcpechoserver.java

import java.net.*;

import java.io.*;

public class tcpechoserver

{

    public static void main(String[] arg) throws IOException

    {

        ServerSocket sock = null;

        BufferedReader fromClient = null;

        OutputStreamWriter toClient = null;

        Socket client = null;

        Try

        {

            sock = new ServerSocket(4000);
```

```

System.out.println("Server Ready");

client = sock.accept();

System.out.println("Client Connected");

fromClient = new BufferedReader(new InputStreamReader(client.getInputStream()));

toClient = new OutputStreamWriter(client.getOutputStream());

String line;

while (true)

{

line = fromClient.readLine();

if ( (line == null) || line.equals("bye"))

break;

System.out.println ("Client [ " + line + " ]");

toClient.write("Server [ "+ line +" ]\n");

toClient.flush();

}

fromClient.close();

toClient.close();

client.close();

sock.close();

System.out.println("Client Disconnected");

}

catch (IOException ioe)

```

```

{

System.err.println(ioe);

}

}

}

//TCP Echo Client--tcpechoclient.java

import java.net.*;

import java.io.*;

public class tcpechoclient

{

public static void main(String[] args) throws IOException

{

BufferedReader fromServer = null, fromUser = null;

PrintWriter toServer = null;

Socket sock = null;

Try

{

if (args.length == 0)

sock = new Socket(InetAddress.getLocalHost(),4000);

else

sock = new Socket(InetAddress.getByName(args[0]),4000);

fromServer = new BufferedReader(new InputStreamReader(sock.getInputStream()));

```



```

fromUser = new BufferedReader(new InputStreamReader(System.in));

toServer = new PrintWriter(sock.getOutputStream(),true);

String Usrmsg, Srvmsg;

System.out.println("Type \"bye\" to quit");

while (true)

{

System.out.print("Enter msg to server : ");

Usrmsg = fromUser.readLine();

if (Usrmsg==null || Usrmsg.equals("bye"))

{

toServer.println("bye");

break;

}

Else

toServer.println(Usrmsg);

Srvmsg = fromServer.readLine();

System.out.println(Srvmsg);

}

fromUser.close();

fromServer.close();

toServer.close();

sock.close();

```

```
}  
  
catch (IOException ioe)  
  
{  
  
System.err.println(ioe);  
  
}
```

SAMPLE OUTPUT

Server:

\$ javac tcpechoserver.java

\$ java tcpechoserver

Server Ready Client Connected Client [hello]

Client [how are you] Client [i am fine] Client [ok]

Client Disconnected

Client : \$ javac tcpechoclient.java

\$ java tcpechoclient

Type "bye" to quit

Enter msg to server : hello

Server [hello]

Enter msg to server : how are you

Server [how are you]

Enter msg to server : i am fine

Server [i am fine]

Enter msg to server : ok

Server [ok]

Enter msg to server : bye

Experiment No:8

PROGRAM USING UDP SOCKET UDP CHAT SERVER/CLIENT

AIM:

To implement a chat server and client in java using UDP sockets.

DESCRIPTION:

UDP is a connectionless protocol and the socket is created for client and server to transfer the data. Socket connection is achieved using the port number. Domain Name System is the naming convention that divides the Internet into logical domains identified in Internet Protocol version 4 (IPv4) as a 32-bit portion of the total address.

ALGORITHM:

Server

1. Create two ports, server port and client port.
2. Create a datagram socket and bind it to client port.
3. Create a datagram packet to receive client message.
4. Wait for client's data and accept it.
5. Read Client's message.
6. Get data from user.
7. Create a datagram packet and send message through server port.
8. Repeat steps 3-7 until the client has something to send.
9. Close the server socket.
10. Stop.

Client

1. Create two ports, server port and client port.
2. Create a datagram socket and bind it to server port.
3. Get data from user.
4. Create a datagram packet and send data with server ip address and client port.
5. Create a datagram packet to receive server message.
6. Read server's response and display it.
7. Repeat steps 3-6 until there is some text to send.
8. Close the client socket.
9. Stop.

PROGRAM

```
// UDP Chat Server--udpchatserver.java

import java.io.*;

import java.net.*;

class udpchatserver

{

public static int clientport = 8040,serverport = 8050;

public static void main(String args[]) throws Exception

{

DatagramSocket SrvSoc = new DatagramSocket(serverport);

byte[] SData = new byte[1024];
```

```

BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
System.out.println("Server Ready");

while (true)
{
    byte[] RData = new byte[1024];

    DatagramPacket RPack = new DatagramPacket(RData,RData.length);

    SrvSoc.receive(RPack);

    String Text = new String(RPack.getData());

    if (Text.trim().length() == 0)

        break;

    System.out.println("\nFrom Client <<< " + Text );

    System.out.print("Msg to Cleint : " );

    String srvmsg = br.readLine();

    InetAddress IPAddr = RPack.getAddress();

    SData = srvmsg.getBytes();

    DatagramPacket SPack = new DatagramPacket(SData,SData.length,IPAddr, serverport);
    SrvSoc.send(SPack);

}

System.out.println("\nClient Quits\n");

SrvSoc.close();

}

}

```

```

// UDP Chat Client--udpchatclient.java

import java.io.*;

import java.net.*;

class udpchatclient

{

    public static int clientport = 8040,serverport = 8050;

    public static void main(String args[]) throws Exception

    {

        BufferedReader br = new BufferedReader(new InputStreamReader (System.in)); DatagramSocket
        CliSoc = new DatagramSocket(serverport);

        InetAddress IPAddr;

        String Text;

        if (args.length == 0)

        IPAddr = InetAddress.getLocalHost();

        Else

        IPAddr = InetAddress.getByName(args[0]);

        byte[] SData = new byte[1024];

        System.out.println("Press Enter without text to quit");

        while (true)

        {

            System.out.print("\nEnter text for server : ");

            Text = br.readLine();

```

```

SData = Text.getBytes();

DatagramPacket SPack = new DatagramPacket(SData,SData.length, IPAddr, clientport );
CliSoc.send(SPack);

if (Text.trim().length() == 0)

break;

byte[] RData = new byte[1024];

DatagramPacket RPack = new DatagramPacket(RData,RData.length);

CliSoc.receive(RPack);

String Echo = new String(RPack.getData()) ;

Echo = Echo.trim();

System.out.println("From Server <<< " + Echo);

}

CliSoc.close();

}

}

```

SAMPLE OUTPUT

Server

\$ javac udpchatserver.java

\$ java udpchatserver

Server Ready

From Client <<< are u the SERVER

Msg to Cleint : yes

From Client <<< what do u have to serve

Msg to Cleint : no eatables

Client Quits

Client

```
$ javac udpchatclient.java
```

```
$ java udpchatclient
```

Press Enter without text to quit

Enter text for server : are u the SERVER

From Server <<< yes

Enter text for server : what do u have to serve

From Server <<< no eatables

Enter text for server : Ok

Experiment No:9

UDP DNS SERVER/CLIENT

AIM:

To implement a DNS server and client in java using UDP sockets.

DESCRIPTION

DNS stands for domain name system. unique name of the host is identified with its IP address through server client communication.

ALGORITHM:

Server

1. Create an array of hosts and its ip address in another array
2. Create a datagram socket and bind it to a port
3. Create a datagram packet to receive client request
4. Read the domain name from client to be resolved
5. Lookup the host array for the domain name
6. If found then retrieve corresponding address
7. Create a datagram packet and send ip address to client
8. Repeat steps 3-7 to resolve further requests from clients
9. Close the server socket
10. Stop

Client

1. Create a datagram socket

2. Get domain name from user
3. Create a datagram packet and send domain name to the server
4. Create a datagram packet to receive server message
5. Read server's response
6. If ip address then display it else display "Domain does not exist"
7. Close the client socket
8. Stop

PROGRAM

```
// UDP DNS Server -- udpdnserver.java

import java.io.*;

import java.net.*;

public class udpdnserver

{

    private static int indexOf(String[] array, String str) { str = str.trim();

    for (int i=0; i < array.length; i++)

    {

        if (array[i].equals(str))

            return i;

    }

    return -1;

}

public static void main(String arg[])throws IOException
```

```

{

String[] hosts = {"yahoo.com", "gmail.com","cricinfo.com", "facebook.com"};

String[] ip = {"68.180.206.184", "209.85.148.19","80.168.92.140", "69.63.189.16"};
System.out.println("Press Ctrl + C to Quit");

while (true)

{

DatagramSocket serversocket=new DatagramSocket(1362);

byte[] senddata = new byte[1021];

byte[] receivedata = new byte[1021];

DatagramPacket recvpack = new DatagramPacket(receivedata, receivedata.length);
serversocket.receive(recvpack);

String sen = new String(recvpack.getData());

InetAddress ipaddress = recvpack.getAddress();

int port = recvpack.getPort();

String capsent;

System.out.println("Request for host " + sen);

if(indexOf (hosts, sen) != -1)

capsent = ip[indexOf (hosts, sen)];

else

capsent = "Host Not Found";

senddata = capsent.getBytes();

DatagramPacket pack = new DatagramPacket(senddata, senddata.length,ipaddress,port);
serversocket.send(pack);

```

```

serversocket.close();

}

}

}

//UDP DNS Client -- udpdnsclient.java

import java.io.*;

import java.net.*;

public class udpdnsclient

{

public static void main(String args[])throws IOException

{

BufferedReader br = new BufferedReader(new InputStreamReader(System.in)); DatagramSocket
clientsocket = new DatagramSocket();

InetAddress ipaddress;

if (args.length == 0)

ipaddress = InetAddress.getLocalHost();

else

ipaddress = InetAddress.getByName(args[0]);

byte[] senddata = new byte[1024];

byte[] receivedata = new byte[1024];

int portaddr = 1362;

System.out.print("Enter the hostname : ");

```

```

String sentence = br.readLine();

Senddata = sentence.getBytes();

DatagramPacket pack = new DatagramPacket(senddata,senddata.length, ipaddress,portaddr);
clientsocket.send(pack);

DatagramPacket recvpack =new DatagramPacket(receivedata,receivedata.length);
clientsocket.receive(recvpack);

String modified = new String(recvpack.getData());

System.out.println("IP Address: " + modified);

clientsocket.close();

}}

```

SAMPLE OUTPUT

Server

```
$ javac udpdnsserver.java
```

```
$ java udpdnsserver
```

Press Ctrl + C to Quit

Request for host yahoo.com

Request for host cricinfo.com

Request for host youtube.com

Client

```
$ javac udpdnsclient.java
```

```
$ java udpdnsclient
```

Enter the hostname : yahoo.com

IP Address: 68.180.206.184

```
$ java udpdnsclient
```

Enter the hostname : cricinfo.com

IP Address: 80.168.92.140

```
$ java udpdnsclient
```

Enter the hostname : youtube.com

IP Address: Host Not Found

RESULT

Thus domain name requests by the client are resolved into their respective logical address using lookup method.

Experiment No:9

ARITHMETIC CALCULATOR USING RPC-RMI

AIM:

To create RMI to perform arithmetic operations using RPC.

DESCRIPTION:

A remote procedure call (RPC) is an IPC that allows a computer program to cause a subroutine or procedure to execute in another address space (commonly on another computer on a shared network) without the programmer explicitly coding the details for this remote interaction. Declare server's remote interface for all calculator operation by extending Remote interface. Define basic calculator operations such as summation, difference, product, quotient and remainder by extending UnicastRemoteObject. An interface is the point of interaction with software or computer hardware, or with peripheral devices such as a computer monitor or a keyboard computing. Some computer interfaces such as a touch screen can send and receive data, while others such as a mouse or microphone, can only send data.

ALGORITHM:

1. Create a remote interface naming Display Int. Each remote method in interface throws a java.rmi.remote exception.
2. Define the class that implements the interface of the remote object extending the class Java.rmi.unicast Remote object.
3. Create the server program, which contains the method used to bind the name to the remote server object.
4. Write the client code. The client program usually consists of simple Java code. Within the client program we have to use lookup method available in java.rmi.
5. Name class to locate a remote object.

6. Compile all java files using javac.
7. Create a stub and skeleton using the compiler rmic Impl filename.
8. Start the RMI registry enabling us to register the server object. RMI registry is started using the command.
9. Run the server program.
10. Run the client program in separate command prompt using command.

PROGRAM:

```
//interface.java
```

```
import java.rmi.*;
```

```
public interface OperationInterface extends Remote
```

```
{
```

```
    public int add(int a,int b)throws RemoteException;
```

```
    public int sub(int a,int b)throws RemoteException;
```

```
    public int mul(int a,int b)throws RemoteException;
```

```
    public int div(int a,int b)throws RemoteException;
```

```
}
```

```
//OperationImpl.java
```

```
import java.rmi.*;
```

```
import java.rmi.server.*;
```

```
public class OperationImpl extends UnicastRemoteObject implements OperationInterface
```

```
{
```

```
    public int add(int a,int b)throws RemoteException
```



```

{

return(a+b);

}

public int sub(int a,int b)throws RemoteException

{ return(a-b); }

public int mul(int a,int b)throws RemoteException

{ return(a*b); }

public int div(int a,int b)throws RemoteException

{ return(a/b); }

public OperationImpl()throws RemoteException

{

super();

}}

//OperationServer.java

import java.rmi.*;

import java.rmi.server.*;

public class OperationServer

{

public static void main(String args[])

{

try {

OperationImpl impl=new OperationImpl();

```

```

Naming.rebind("HelloService",impl);

}

catch(Exception e)

{ System.out.println(e.toString());

}}}

//OperationClient.java

import java.io.*;

import java.rmi.*;

public class OperationClient

{

public static void main(String arg[])throws RemoteException

{

Try

{

int a,b,ch;

String str,str1;

DataInputStream in=new DataInputStream(System.in);

OperationInterface obj = (OperationInterface)Naming.lookup("HelloService");

Do

{

System.out.println("ARITHMETIC OPERATION");

System.out.println("1.ADDITION");

```

```

System.out.println("2.SUBTRACTION");

System.out.println("3.MULTIPLICATION");

System.out.println("4.DIVITION");

System.out.println("5.EXIT");

System.out.println("ENTER YOUR OPTION :");

str1=in.readLine();

ch=Integer.parseInt(str1);

switch(ch)

{

case 1:

System.out.println("Enter the value for A:");

str=in.readLine();

a=Integer.parseInt(str);

System.out.println("Enter the value for B:");

str=in.readLine();

b=Integer.parseInt(str);

System.out.println("ADD="+obj.add(a,b));

break;

case 2:

System.out.println("Enter the value for A:");

str=in.readLine();

a=Integer.parseInt(str);

```

```
System.out.println("Enter the value for B:");

str=in.readLine();

b=Integer.parseInt(str);

System.out.println("SUB="+obj.sub(a,b));

break;

case 3:

System.out.println("Enter the value for A:");

str=in.readLine();

a=Integer.parseInt(str);

System.out.println("Enter the value for B:");

str=in.readLine();

b=Integer.parseInt(str);

System.out.println("MUL="+obj.mul(a,b));

break;

case 4:

System.out.println("Enter the value for A:");

str=in.readLine();

a=Integer.parseInt(str);

System.out.println("Enter the value for B:");

str=in.readLine();

b=Integer.parseInt(str);

System.out.println("DIV="+obj.div(a,b));
```

```
break;

}

}while(ch!=5);

}

catch(Exception e)

{

System.out.println(e.toString());

}}}
```

SAMPLE OUTPUT

```
C:\>cd Arithmetic op

C:\Arithmetic op>javac *.java

C:\Arithmetic op>rmic OperationImpl

C:\Arithmetic op>start rmiregistry

C:\Arithmetic op>start java OperationServer

C:\Arithmetic op>java OperationClient

ARITHMATIC OPERATION

1.ADDITION

2.SUBTRACTION

3.MULTIPLICATION

4.DIVITION

5.EXIT

ENTER YOUR OPTION :
```

1

Enter the value for A:

3

Enter the value for B:

5

RESULT:

Thus the RMI program for performing arithmetic operations such as addition, subtraction, multiplication and division is created and the output is verified.